

Building React Native Apps in 2025: Best Practices Guide

React Native continues to evolve rapidly in 2025, offering robust support for both iOS and Android development from a single codebase. This guide compiles the most up-to-date practices for building high-quality React Native apps, emphasizing smooth animations, slick UI/UX, modern development workflows, and enterprise-level considerations. We'll cover both **Expo (Managed)** and **React Native CLI (Bare)** approaches, highlight recommended libraries (like Reanimated, Moti, Lottie), and discuss architecture patterns, TypeScript, testing, performance, deployment, and backend integration.

Development Workflow: Expo vs React Native CLI

React Native projects can be developed using **Expo's managed workflow** or the **React Native CLI (bare workflow)**. The choice impacts how you handle native code, build processes, and available tooling. Below is a comparison of key aspects of Expo and React Native CLI in 2025:

Aspect	Expo (Managed Workflow)	React Native CLI (Bare Workflow)
Getting Started	Easiest setup – no Xcode/Android Studio needed for development. Use <code>npx create-expo-app</code> to bootstrap a project and run with the Expo Go app (scan a QR code to load the app on device) ¹ ² . Expo abstracts native configurations, letting you focus on JavaScript.	Requires full native toolchain. Use <code>npx react-native init</code> to create a project, then run via Xcode (iOS) or Android Studio/Gradle (Android). Deeper native setup is needed (installing Xcode, Android SDK) but you gain direct control ³ .
Native Module Access	Limited to Expo-supported native modules by default. Expo includes many APIs out-of-the-box (camera, sensors, etc.) and now supports adding custom native modules via Config Plugins without fully ejecting ⁴ ⁵ . Some very low-level modules may still be unavailable until added via a development build.	Full access to all native modules and device APIs. You can install any third-party React Native library or write custom native code as needed. No restrictions – ideal if your app relies on specialized or unsupported native features ⁶ ⁷ .

Aspect	Expo (Managed Workflow)	React Native CLI (Bare Workflow)
Build & Deployment	Uses Expo Application Services (EAS) for cloud builds and deployment. You can build app binaries in the cloud with one command (e.g. <code>eas build -p ios</code> or <code>eas build -p android</code>), handling signing and app store submission automatically ⁸. Expo's EAS Submit can upload builds to App Store Connect or Google Play Console ⁸. Over-the-air (OTA) updates are built-in via expo-updates/EAS Update, letting you push JS and asset updates instantly without waiting for app store review ⁸ ⁹.	Uses native build tools or your own CI setup. iOS apps are archived and signed via Xcode or Fastlane; Android apps are built with Gradle (<code>./gradlew assembleRelease</code> for APK/AAB). You manually handle provisioning profiles, keystores, and store submissions (or script it via Fastlane, Bitrise, GitHub Actions, etc.). OTA updates are <i>not</i> built-in – you'd use Microsoft CodePush or similar for live updates if needed ¹⁰.
Development Experience	Live reloading and debugging through Expo CLI (<code>expo start</code>). Expo Go app allows running your app on devices instantly. Physical device testing is simple – just scan a QR code or use the Expo app ² ¹¹. Expo abstracts away Metro bundler config and offers a unified dev menu.	Development uses Metro bundler as well (via <code>npx react-native run-ios/android</code>), but you'll need simulators or devices connected via USB. Debugging is done with the React Native dev menu, React DevTools, or Flipper. Hot Reload/Fast Refresh works similarly. However, tasks like linking native libraries (auto-linking or CocoaPods) are your responsibility.
App Size & Performance	Expo apps historically had slightly larger binary size and included some unused native code (since they bundle the Expo runtime). Recent improvements (including Hermes engine by default and support for the New Architecture) have narrowed the performance gap ¹². Still, a managed Expo app might be a bit larger due to built-in modules, but it achieves near-native performance, even for animations and complex UI ¹².	Bare apps include only what you add. You have full control to keep the app lean – e.g., exclude libraries you don't need. This can mean smaller binary size and potentially faster startup. Performance can be tuned at a low level (enabling Hermes, using the New Architecture, optimizing native code). In practice, both workflows can achieve 60fps smooth apps, but CLI gives more room for custom optimizations ¹³ ¹⁴.

Aspect	Expo (Managed Workflow)	React Native CLI (Bare Workflow)
Use Cases	Recommended for rapid prototyping, solo/small teams, or projects where you want to maximize development speed and avoid native complexity. In 2025, Expo is production-ready even for complex apps, thanks to support for native modules via config plugins and the New Architecture ⁴ ⁵ . It simplifies maintenance (with OTA updates and managed build processes) and reduces the need for platform-specific expertise.	Recommended for apps that require extensive native customizations or integration of libraries that Expo doesn't support out-of-the-box. Also suitable for large-scale or enterprise apps where fine-grained control over the app's native layers, build configuration, and performance is necessary ⁷ ¹⁵ . If your team has native iOS/Android experience or your app demands it (e.g. custom native UI components, background services), CLI is the better fit.

Both workflows are continually improving. Notably, **React Native's New Architecture** (Turbo Modules and Fabric renderer with JSI) is supported in both Expo (SDK 52+ enables it by default) and bare RN projects, bringing performance boosts and closer-to-native feel ¹⁶ ¹⁷. In fact, libraries like React Native Reanimated v4 **require** the New Architecture and latest RN versions going forward ¹⁸ ¹⁹. Expo has embraced this: as of SDK 53, all Expo modules support the New Architecture, and most new Expo apps have it enabled by default ¹⁷ ²⁰.

Bottom line: Expo provides a smooth, high-level DX (developer experience) with managed services, whereas React Native CLI gives you ultimate flexibility. Many teams start with Expo for speed, then “eject” (or transition to bare workflow) only if necessary. Thanks to EAS and config plugins, ejecting is far less common now since Expo can handle most native needs **without** giving up the managed workflow ⁵.

Animations and High-Performance UI

A standout feature of modern mobile apps is delightful, **butter-smooth animations** and a polished UI. In 2025, React Native's animation ecosystem has matured to deliver 60+ FPS interactions on both platforms. Key tools and libraries include:

- **React Native Animated API (Built-in):** React Native's default `Animated` API allows timing, spring, and decay animations. It's declarative but runs animations on the JavaScript thread, which can cause jank if the JS thread is busy ²¹ ²². For simple UI tweaks (fades, slides) it's workable, but it struggles with very complex animations or gestures due to the JS bridge overhead.
- **React Native Reanimated 2/3/4: Reanimated** is the state-of-the-art animation library for RN, designed for **high-performance** and complex interactions. It **offloads animation work to the native UI thread**, bypassing the JS thread bottleneck ²². This means even with heavy JS workload, animations stay fluid at 60 FPS. Reanimated provides a **declarative hooks API** (`useSharedValue`, `useAnimatedStyle`, etc.) and seamless **gesture integration** via `react-native-gesture-handler` ²³ ²⁴.

Why Reanimated? The difference is stark when compared to the default Animated API, as shown below:

Feature	React Native Animated (Core)	React Native Reanimated
Thread	Runs on JS thread (possible lag)	Runs on Native UI thread (smooth) ²² ²⁴
Performance	Limited by JS-native bridge	No bridge traffic; 60 FPS even for complex animations ²¹ ²⁴
Syntax	Imperative (callback-heavy)	Declarative hooks & worklets (more readable) ²² ²⁵
Gesture Support	Basic support	Deep integration with Gesture Handler for natural interactions ²⁶ ²⁷
Debugging Tools	Limited	Advanced logging & dev tools (Inspector)

Reanimated (currently at v3 with v4 in beta) introduces powerful features: **worklets** (write JS that runs on the UI thread), **shared values** (state that drives animations), and hooks like `useAnimatedStyle` to tie animations to component styles ²⁸ ²⁹. For example, to animate a fade-in with Reanimated:

```
import Animated, { useSharedValue, useAnimatedStyle, withTiming } from 'react-native-reanimated';

const opacity = useSharedValue(0); // shared value for opacity
const animatedStyle = useAnimatedStyle(() => ({
  opacity: opacity.value,
}));
// Trigger animation (runs on UI thread because of the 'worklet' directive)
const fadeIn = () => {
  'worklet';
  opacity.value = withTiming(1, { duration: 300 });
};
// ... in JSX:
<Animated.View style={[styles.box, animatedStyle]} onLayout={fadeIn} />
```

In this snippet, `opacity.value` is updated inside a worklet, so the interpolation to 1.0 happens natively without blocking JS. Reanimated also offers **physics-based animations** like `withSpring` and `withDecay` for bouncy or natural motion ³⁰ ³¹, and even supports **shared element transitions** (seamless screen-to-screen animations) to implement UI effects like those in Instagram or Pinterest ³².

Reanimated v4 (expected stable in 2025) adds support for **CSS animations and transitions** on native elements, making web-like animations easier to implement ³³. It remains backward-compatible with v3, but note that Reanimated 4.x requires the New Architecture (no support for the old RN bridge) ¹⁹.

- **Moti**: Built on Reanimated, **Moti** provides a *simplified, high-level API* for animations. Think of it as the React Native equivalent of Framer Motion. With Moti, you can simply declare initial, animate, and exit

states for a component, and it will handle the rest (mounting/unmounting animations, transitions, etc.) ³⁴ ³⁵ . For example:

```
import { MotiView } from 'moti';
<MotiView
  from={{ opacity: 0, scale: 0.9 }}
  animate={{ opacity: 1, scale: 1 }}
  exit={{ opacity: 0 }}
  transition={{ type: 'spring', damping: 10 }}
/>
```

This one component replaces manual use of Reanimated hooks for many cases. **Highlights of Moti** include: 60 FPS native-thread animations (since it uses Reanimated under the hood), an intuitive props-driven API, support for **variants/keyframes**, and built-in **AnimatePresence** for coordinating enter/exit animations across screens ³⁵ ³⁶ . Moti is great for developers who want fluid animations without wrestling with lower-level worklets. (Under the hood, it's using Reanimated's API, so it inherits the performance benefits ³⁷ .)

- **Lottie:** For **complex pre-made animations** (like those created in Adobe After Effects), **Lottie** is the go-to solution. Lottie loads animation definitions (JSON files) and renders them natively as vector graphics. This is perfect for things like loading spinners, onboarding illustrations, or any custom motion graphic provided by your design team. Using Lottie in RN (via `lottie-react-native`) is straightforward – include a `<LottieView source={require('./anim.json')} autoPlay loop />` component in your render. The animations run at 60 FPS and are resolution-independent. Best practices in 2025 include using the newer `.lottie` binary format for smaller file sizes and faster loading ³⁸ , and lazy-loading the animation files (don't bundle huge JSON files that might not be immediately needed ³⁹). Lottie is **cross-platform** and now well-supported in Expo (via the `expo-av` or `lottie-react-native` with config plugin).
- **Gesture Handler & Gesture APIs:** Smooth, high-end UI often involves touch **gestures** (swipes, drags, pinch-zoom, etc.). The **React Native Gesture Handler** library is a must for handling complex gestures without lag. It works seamlessly with Reanimated – for example, you can define a pan gesture and update shared values on the UI thread as the user moves their finger ⁴⁰ ⁴¹ . This combination enables effects like swipeable cards, interactive animations, and responding to gestures with zero jank. In Reanimated 3, the new `Gesture` API from `react-native-gesture-handler` can be used to define gestures declaratively and chain them (e.g., `.onStart`, `.onUpdate`, `.onEnd` handlers) ⁴⁰ ⁴¹ . Always prefer these native-driven gestures over React's touch responders for best performance.
- **Skia and 3D Graphics:** For truly custom, high-end graphics (beyond typical view animations), the **React Native Skia** library provides a 2D drawing canvas accelerated by the GPU. Skia can render shapes, shaders, and even real-time effects at high performance, and it's increasingly used for custom charts or graphics-heavy screens. It integrates with Reanimated for driving animations on the canvas. Additionally, **Three.js** and **Expo GL** can be used for 3D content in RN apps (for example, the Starlink mobile app uses Three.js with Expo's WebGL to render interactive star maps ⁴²). These are specialized use-cases, but worth noting if your app demands **custom visualization** or game-like

graphics. With the New Architecture improving throughput and libraries like Skia adding WebGPU support, 2025 is seeing more feasible high-end graphics in RN ⁴³ .

Tip: When implementing animations, use the **Performance Monitor** (in the RN dev menu or Flipper) to watch your FPS. Jank (frame drops) usually indicate work on the JS thread. By leveraging Reanimated (or libraries built on it), you ensure most animation work stays on the UI thread. Also remember to use the **Hermes JS engine** (more on this in Performance section) – Hermes can improve animation smoothness by making JavaScript execution faster and more predictable, thus avoiding hitches that could disturb animations ⁴⁴ .

State Management and Architecture Patterns

A well-structured app architecture is key to scalability and maintainability. In 2025, React Native developers often combine **modern state management libraries** with proven architectural patterns to separate concerns. Here are recommended approaches:

- **Redux Toolkit (RTK) – Global State Management:** Redux remains a popular choice for managing global app state, especially for complex or large applications. **Redux Toolkit** is the standard way to use Redux now – it reduces boilerplate with `configureStore` and `createSlice`, and includes powerful addons like **RTK Query** for data fetching and caching. Redux enforces a **unidirectional data flow** (state is updated via dispatched actions and pure reducers), which can make app behavior predictable. In an RN app, you'd set up a Redux store (often in an `store.ts` using RTK's `configureStore`), wrap your app in `<Provider store={store}>`, and use hooks like `useSelector` and `useDispatch` in components. Redux Toolkit encourages **immutable** state updates (under the hood it uses Immer), and it integrates well with TypeScript. *When to use Redux:* when you have substantial shared state or complex view logic that needs to be centrally managed (e.g., user authentication info, a cart in an e-commerce app, etc.), especially across many screens. In an enterprise context, using Redux or a similar centralized store aligns with clean architecture (separating business logic from UI) ⁴⁵ .
- **MobX – Observable State and MVVM:** MobX offers an alternative approach using observable state. You define plain classes or objects and decorate them (using `makeAutoObservable` or similar) so that any changes to their properties automatically update the UI. This feels closer to an MVVM (Model-View-ViewModel) pattern – your “ViewModel” is a MobX store (class) with observable properties and actions, and your React components become observers of that state. MobX requires far less boilerplate than Redux; you simply mutate state and MobX handles the reactivity (it will trigger component re-renders where those observables are used). This can be very productive for apps with complex domain models. **When to use MobX:** if you prefer an object-oriented state management and need to manage complex data models with relationships, or find Redux too verbose. It's important to structure MobX stores well to avoid unwieldy global mutable state – many devs still compartmentalize by feature or domain (e.g., an `UserStore`, `ProductStore`, etc.). MobX can fit into MVVM: each screen can have an associated state object (ViewModel) that the view observes.
- **Context API (React Context) – Lightweight State & Dependency Injection:** For **small apps or theme/settings data**, the built-in Context API can be sufficient. React Context provides a way to

pass state through the component tree without prop drilling. For example, you might use Context to handle a current User object, app theme (dark/light mode), or a simple form of navigation or configuration state. The Context API combined with `useReducer` can mimic a mini-Redux (indeed, Redux is partially implemented with context under the hood). However, Context re-renders can become costly if not managed (it re-renders all consumers when value changes). It's best for *low-frequency updates* (like a theme toggle) or data that truly needs to be global. In 2025, many apps use a mix: Context for a few global concerns, and a library like Redux or MobX for more complex state. One common pattern is using Context to provide instances of your stores or services (e.g., provide a Redux store or MobX stores via Context at the root, so all components can access them).

- **Clean Architecture and MVVM Patterns:** Beyond state libraries, it's important to organize your code in logical layers. A recommended practice is to separate: **UI layer, Presentation layer, and Business Logic layer** ⁴⁶ ⁴⁷ . In a React Native context:

- *Display/UI layer:* Your presentational **components** (often functional components) that render views. These should ideally be stateless UI pieces that simply receive props and display them. Styles and layout code reside here. By keeping these components dumb, you can reuse them and test them easily.
- *Presentation (ViewModel) layer:* This is where you handle state and user interactions for a screen or component. In React, these often take the form of **custom Hooks** or container components. For example, a `useAuthentication()` hook might encapsulate all logic for logging in/out, and provide state (like `user`, `isLoading`) plus actions (like `login`, `logout`) to the UI. This layer interacts with the business logic (e.g., calls an API or uses a service) and updates state accordingly. In MVVM terms, these are the **ViewModels** – they serve data to the View and handle user commands.
- *Business Logic layer:* This includes data fetching, network calls, local database access, and any complex computations. Ideally, this logic is encapsulated in **services or repository modules** (plain TypeScript classes or utility modules) that the presentation layer can call. For instance, an `AuthService.login(credentials)` method that talks to Firebase or your backend, returning a user token. Business logic may also include state management stores (Redux reducers or MobX stores can be thought of as part of this layer, managing the state that comes from network or other sources).

By structuring this way, you achieve *separation of concerns*: UI components don't directly fetch data; they rely on a ViewModel or global state which in turn uses services. This makes testing easier – you can unit test business logic functions without any UI, and you can use mock data for testing UI components. It also improves scalability: multiple screens can share the same business logic or global state. For example, two different screens might both use the `ProductStore` or call `ProductService.fetchProducts()`, but display the data differently. Clean architecture in RN often goes hand-in-hand with TypeScript interfaces to define contracts between layers (e.g., an interface for your service methods, or types for the data models).

Example: A feature-based folder structure might reflect this: `features/ShoppingList/` containing `ShoppingListScreen.tsx` (UI), `useShoppingList.ts` (hook that uses, say, a Redux slice or service), and `shoppingListService.ts` (business logic for data fetching, perhaps using a REST API). This keeps each feature self-contained and modular ⁴⁸ ⁴⁹ .

- **Other Patterns:** Some developers apply classic MVC or MVP patterns in RN, but those are less common than MVVM style with hooks. **Redux (Flux architecture)** itself is a pattern (one-way data

flow with a central store) and can be combined with MVVM – for instance, your ViewModel hook might internally use Redux state/selectors. The key is to avoid tightly coupling your UI to data sources. By decoupling, you can swap implementations (e.g., replace a REST API with a GraphQL service) without rewriting UI components.

In summary, choose a state management solution that fits your app's complexity: **Redux Toolkit** for predictable large-scale state management with good dev tools (Redux DevTools, time-travel debugging), **MobX** for quick and easy observable state (great for complex forms or rich domain models), or Context for simple needs. Use an **architecture pattern like MVVM or Clean Architecture** to keep your code organized – this decouples the *what* (UI) from the *how* (business logic) ⁴⁵ and makes testing and maintaining the app much easier.

Using TypeScript in React Native

TypeScript has become the default for new React Native projects by 2025, bringing static typing to catch errors early and improve code quality. Both Expo and React Native CLI make it easy to use TypeScript:

- **Project Setup:** New React Native CLI projects target TypeScript by default (since RN 0.71+) ⁵⁰. If you use `npx react-native init MyApp`, the template comes with TypeScript configured. Expo projects can be created with `npx create-expo-app@latest --template` (choose the TypeScript template) or simply by adding a `*.ts` or `*.tsx` file to an existing project – Expo will detect it and prompt to install the TypeScript compiler and config automatically ⁵¹. In short, starting a RN project in 2025 usually means starting with TS.
- **Adding TS to Existing Apps:** If you have a plain JS React Native app and want to add TypeScript, the steps are straightforward: install the TypeScript compiler and type definitions (`npm install --save-dev typescript @types/react @types/react-native @types/jest`), then add a `tsconfig.json` at the root extending React Native's TS config preset ⁵² ⁵³. For example, a minimal `tsconfig.json` can just contain:

```
{ "extends": "@react-native/typescript-config" }
```

This preset sets up sensible defaults for RN. Rename your files from `.js` / `.jsx` to `.ts` / `.tsx` (for files containing JSX). You can then run `npx tsc --noEmit` to check types without producing build output ⁵⁴ ⁵⁵ (the actual app bundling is still done by Metro/Babel, which strips type annotations).

- **Type Definitions:** React Native's core types (for components like `Text`, `View`, etc.) are provided by the React Native DefinitelyTyped package, which is usually installed by default. When using libraries, install their type definitions if needed (e.g., `@types/react-navigation` if a library doesn't ship its own types). In 2025, most popular RN libraries are written in TS or include type defs.
- **Typing Components:** Define **Prop types** and **State types** for your components. In function components, you can use `React.FC<Props>` or simply annotate the props parameter. For example:


```
type GreetingProps = { name: string; count?: number };
const Greeting: React.FC<GreetingProps> = ({ name, count = 1 }) => {
  return <Text>Hello {name}{'!'.repeat(count)}</Text>;
};
```

This ensures that when someone uses `<Greeting name="Alice" count={5} />`, they pass correct types. For class components (less common now), you'd use `Component<Props, State>` generic typing ⁵⁶ ⁵⁷.

- **Strictness and Linting:** It's recommended to enable strict mode in TS (`"strict": true` in `tsconfig`) for maximum type-safety. Also consider using ESLint with TypeScript rules (the React Native Community ESLint config supports TS). This will catch issues like unused variables, wrong types, etc., during development.
- **React Native and Babel:** Under the hood, RN uses Babel to transpile your TypeScript (strip types) when bundling. The TypeScript compiler (`tsc`) is typically used only for type checking, not for emitting the final JS in RN projects ⁵⁸. This means your build process remains fast (Babel handles it), but you might occasionally need to configure Babel for TS path aliases or experimental features. If you want to use **path aliases** (e.g., `import {X} from "@components/X"`), you'll need to configure both your `tsconfig.json` (with `paths`) and Babel (with a plugin like `module-resolver`) ⁵⁹.
- **Expo and TS:** Expo projects handle TypeScript seamlessly. By including a `tsconfig.json` or simply using TS files, Expo will ensure the Metro bundler knows how to compile them. Expo's recommended config is similar, extending the community TS config. In Expo, when you do an OTA update or EAS build, your TypeScript is compiled just like in bare RN.

The benefits of TypeScript are significant: you catch errors (like mistyping a prop name or misusing a module) at compile time, get autocompletion and refactoring support in IDEs, and can self-document your code with interfaces and types. For example, defining an interface for a data model (like `interface Product { id: string; name: string; price: number; }`) means every part of your app can reliably use `Product` objects with correct properties.

Tips: Leverage advanced TS features for robustness – e.g., use **Enums or Literal Types** for fixed options instead of strings, utilize **Union types** for different state shapes (like a union for possible screen params), and consider **Generics** for reusable functions (such as a utility hook that works with any data type). Also, use the **React Navigation** library's TypeScript support to type your navigation routes, so you get compile-time checks on route names and params. With the entire React Native ecosystem moving toward TypeScript (even Metro's config and ESLint presets assume TS), adopting it will significantly improve your development experience and reduce runtime bugs.

Testing: Unit, Integration & E2E

Robust testing ensures your React Native app works correctly across the wide range of devices and scenarios in production. A comprehensive testing strategy in 2025 includes **unit tests**, **integration tests**, and **end-to-end (E2E) tests**:

- **Unit Testing with Jest:** Jest is the de facto unit testing framework that comes bundled with React Native. It allows you to test JavaScript functions, React components, and Redux slices in isolation. For example, you might write a test for a utility function (pure logic) or use **React Test Renderer / React Native Testing Library** to render a component in a test environment. Jest supports **snapshot testing** as well – capturing a rendered output and comparing it later to detect changes, although snapshots should be used sparingly (and kept small) to be effective ⁶⁰ ⁶¹. A simple unit test might assert that a Redux reducer returns the expected new state given an action, or that a pure function formats a date correctly. These tests run in Node.js, so no actual mobile device is needed.
- **Component & Integration Testing:** To test how components behave *in concert* – e.g., ensuring a component renders correctly given some context or that pressing a button triggers the right state changes – you can use **React Native Testing Library (RNTL)**. This library (built on top of React Test Renderer) lets you render components, simulate user interactions, and make assertions on the UI output ⁶² ⁶³. For instance, you can render a `<GroceryShoppingList />` component, use `fireEvent.changeText()` to type into a TextInput, then `fireEvent.press()` on a Button, and finally assert that a new item appears in the list ⁶⁴ ⁶⁵. RNTL encourages testing from the user's perspective – query by text or accessibility labels, not by internal component instances, which leads to more resilient tests ⁶³. These tests are “integration” tests in the sense that they might involve multiple components together (and possibly involve a fake Redux store or context provider to supply data). They still run in a simulated environment (Jest's JS DOM or an equivalent), not on a real device, but they can catch a lot of UI logic issues. Note that because these tests don't run the actual iOS/Android code (just the JS), they won't catch native module issues, but they are fast and great for logic and layout checks ⁶⁶.
- **End-to-End (E2E) Testing with Detox:** E2E tests execute your app in an emulator/simulator or on a real device, interacting with it like an end user. **Detox** (by Wix) is a gray-box testing framework tailored for React Native apps. It launches the app, allows you to simulate taps, swipes, entering text, etc., and can assert that certain elements appear or certain conditions are met. For example, you can write a test that goes through a full login flow: start the app, fill in a username and password field, tap "Login", then expect to see a logout button on the next screen. Detox is known for its speed and reliability in RN testing and supports both iOS and Android. It works with Jest as the test runner by default ⁶⁷. Setting up Detox involves configuring it with your build system – essentially, you build a debug version of your app with special Detox hooks, then run the tests which drive that app. In 2025, Detox and similar tools can even integrate with CI pipelines so that your E2E tests run on each commit (using emulators in a CI environment). E2E tests give you confidence that the whole app (JS + native layers + device permissions, etc.) works together correctly. They can catch platform-specific bugs or integration issues that unit tests might miss.
- **Testing Strategy:** A good strategy is the “testing pyramid”: have a solid base of unit tests (fast, covering many edge cases of your business logic), a layer of integration tests for critical components and flows, and a smaller number of E2E tests covering the core user journeys. For instance, unit test

your Redux reducers and utility functions thoroughly. Use integration tests to verify that a form component reacts correctly to user input (without firing up a device). Then use E2E to test something like “a user can sign up, log in, and see their profile” end-to-end. This layered approach ensures quick feedback (unit tests run in seconds) and broad coverage, with the slower E2E tests reserved for full workflows.

- **Tools and Best Practices:**

- Use the **Jest CLI** (`jest --watch`) during development to run tests on file changes. Jest supports mocking modules – e.g., you can mock `react-native/Libraries/Animated` to a no-op if needed, or mock network calls using `jest.mock('axios')` to return preset data. This is useful for unit tests to isolate the code under test.
- For React Native Testing Library, prefer queries like `getByText`, `getByA11yLabel` (accessibility label) to find elements, as this simulates how users find content ⁶³. Avoid querying by test IDs unless necessary; testing library encourages testing what the user sees.
- Utilize **Jest's setup files** to configure any polyfills or environment setup (for example, setting up `jest.useFakeTimers()` if you have animations or timers to control).
- **Snapshot tests:** You can use `toMatchSnapshot()` on a rendered component output to detect unexpected changes. Just be cautious – update snapshots only when changes are intended, and keep snapshot files small ⁶⁰ ⁶⁸.
- **Detox practices:** Organize your E2E tests by flows or screens, use the `by.id()` or `by.text()` selectors to find elements (Detox can match React Native `testID` props or text). Detox has synchronization built-in (it waits for animations and network calls by default), but sometimes you might need to add explicit waits or disable animations in test builds for consistency.
- Consider **mocking external services** in E2E – e.g., if your app calls a backend API, you might run a mock server or stub network responses during testing, so tests don't rely on real backend (for consistency and speed).
- **CI integration:** Ensure your CI (Continuous Integration) pipeline runs your test suites. For unit/integration tests, this is straightforward (just run `npm test`). For Detox, you can set up headless emulators; there are guides and even Expo has documentation for running E2E on their build infrastructure ⁶⁹.

By following these testing best practices, you aim to catch issues early. As a quick summary from an enterprise perspective: **unit tests** validate individual functions and modules, **integration tests** cover interactions between components/modules, and **E2E tests** simulate real user behavior on real devices ⁷⁰. Embracing all three levels will significantly improve your app's reliability and help you catch regressions when adding new features.

Performance Optimization Techniques

Performance is critical for retaining users. A React Native app, if optimized well, can feel as snappy as a fully native app. Here are key performance optimization tips and best practices in 2025:

- **Enable Hermes Engine:** Hermes is a lightweight JavaScript engine optimized for React Native. It reduces app startup time, memory usage, and improves overall execution speed ⁴⁴. As of RN 0.64+, Hermes is often enabled by default (and Expo enables Hermes in production builds by default). Verify it's on by checking `HermesInternal` in global or looking at your console logs on app start (it

will mention Hermes). If not, enable it in your `android/app/build.gradle` (`enableHermes: true` in `react` block) and for iOS, ensure you use the Pod for Hermes. Hermes works by ahead-of-time compiling JS to bytecode, so parsing is faster ⁷¹. The result: faster launch and less jank during heavy JS operations. It's practically a must-use for RN apps now.

- **Lazy Load Code and Components:** Avoid loading all your JavaScript at startup. Split your app so that rarely-used screens or heavy modules are loaded on demand. In React (Native), you can use `React.lazy()` and `Suspense` to code-split components. For example, if you have an “Settings” screen not needed on launch, you can do: `const SettingsScreen = React.lazy(() => import('./SettingsScreen'));`. When that screen is navigated to, RN will load it. This ensures initial bundle size is smaller, reducing launch time ⁷² ⁷³. Additionally, Metro (the RN bundler) has a feature called **inline requires** (enabled by default in recent versions) which automatically defers requiring modules until they're used. This acts as a form of lazy loading to speed up startup. To leverage this, keep your imports at top of files (Metro will transform them to lazy require calls where beneficial) ⁷⁴ ⁷⁵. The RN docs specifically recommend lazy-loading **large screen components** to improve startup ⁷³. Just be careful that any module with side effects is loaded appropriately (avoid heavy side effects at import time, as those negate lazy loading benefits ⁷⁶).
- **Optimize and Cache Images:** Loading big images can be memory-intensive. Use appropriate image sizes and formats. For example, prefer JPEG/WEBP for photos (WEBP support is now built-in on Android and iOS14+), and PNG/SVG for graphics. Use **thumbnail previews** or blurred low-res placeholders for large images to improve perceived performance. Implement caching – the `Image` component on iOS caches images automatically; on Android you might use a library like `react-native-fast-image` for aggressive caching and to avoid re-fetching images. Also, **lazy-load images** that are off-screen (e.g., in a long list) by using FlatList's lazy loading or intersection observers via libraries. If using Expo, the `expo-image` component (introduced as a performant image component) is a good replacement for the default Image for better caching and performance.
- **FlatList and large lists:** When rendering long lists, always use **FlatList** or `SectionList` instead of scrolling through huge `ScrollViews`. FlatList is highly optimized – it virtualizes items (only renders what's on screen). Configure your FlatList with proper `keyExtractor`, and consider using `getItemLayout` if item heights are fixed to enable skip indexing (makes scroll-to-index faster) ⁷⁷. In 2025, **FlashList** (by Shopify) has gained popularity – it's a drop-in replacement for FlatList with improved performance for extremely large lists (it uses smart measurement and is written in the new architecture for efficiency). If your app has feeds or large tables, FlashList can scroll more smoothly with less memory usage. Remember to **paginate** your data – don't load 1000 items at once if not needed. Use the `onEndReached` of FlatList to load more when nearing the end.
- **Minimize Re-renders:** Unnecessary re-renders can slow your app, especially in lists or complex screens. Use **memoization** techniques: `React.memo` for functional components to prevent re-renders when props haven't changed, and the hooks `useMemo` and `useCallback` to avoid recreating functions or values on every render unless dependencies change ⁷⁸. For instance, if you have a big list item component, wrap it in `React.memo` so it doesn't re-render unless its props (e.g., item data) change. Use `useCallback` for event handlers (press handlers, etc.) so that child components don't see new props on every parent render. Also, avoid putting a lot of state in one component such that any change causes a large subtree to render – instead, split state logically and

lift state down closer to where it's used when possible. The React DevTools can highlight re-renders; use it to detect if a component re-renders too often.

- **Avoid expensive computations on the JS thread:** Heavy calculations or JSON parsing in JavaScript can jank the UI. If you have to do a large computation (say, parsing a big data file, or image processing), consider offloading it: you can use the **JSI**/native modules (write native code for heavy work), spawn a WebWorker thread (limited support in RN), or simply perform it in small chunks spread over frames using `InteractionManager` or `setTimeout` to yield occasionally. The key is to keep the JS event loop free to handle UI updates. Hermes helps here by being faster at JS, but you might still need to break down very heavy tasks.
- **Memory optimization:** Mobile devices have limited RAM. Watch for **memory leaks** – e.g., lingering timers or subscriptions. Always clear intervals/timeouts in `useEffect` cleanup, and remove event listeners on unmount. If you use a lot of large objects or arrays, be mindful of holding references longer than needed. The new **Fabric** renderer in RN (part of New Architecture) is more efficient with memory and GC, but your JS usage matters too. Use tools like Xcode's Instruments (Allocations) or Android Studio Profiler to monitor memory if your app is memory intensive (like dealing with many images or large data sets).
- **Leverage the New Architecture:** React Native's new architecture (Fabric & TurboModules) reduces the overhead of the bridge by allowing direct JS-to-native calls and synchronously accessing native views. This can improve performance for certain operations (especially UI updates and interop-heavy logic). If you enable the new architecture (it's on by default for new projects in SDK 52+ in Expo, and optional in RN CLI 0.70+ but will become the default late 2025⁷⁹²⁰), you may see smoother interactions and lower latency. Libraries like Reanimated v4 and FlashList are optimized for the new architecture and drop support for the old one¹⁸. Therefore, migrating to the new architecture when possible can future-proof your app and unlock performance improvements. Expo makes it easy – just ensure you're on the latest SDK and it's mostly handled automatically¹⁷²⁰. In a bare app, you'd enable it by adjusting build flags (`RNNI_ENABLED` etc., as per RN docs).
- **Profiling and Monitoring:** Use **Flipper** (the official RN debugging tool) which has plugins for performance monitoring. Flipper can show you FPS, CPU usage, memory usage in real time. The React DevTools profiler can help identify slow rendering paths in your React component tree. For instance, if a render is taking a long time, you might discover a heavy function running in render – you could move that to `useEffect` or optimize it. Also, keep an eye on **anonymous function props** passed to child components (they cause child to re-render each time).
- **Release build optimizations:** Remember that the performance of a Debug build (especially in Expo Go or with dev mode on) is much worse than a release build. Always test performance on **release mode** on actual devices. Release builds have optimizations like minification and dead-code elimination. If using ProGuard/R8 on Android (default), ensure you've optimized native libraries configuration (usually RN template does it for you).
- **Other Tips:**
- **Batch updates:** In React 18, state updates are batched even in promises and event handlers by default, which reduces unnecessary re-renders. Utilize that by doing related state updates together.

- **Windowed list rendering:** For extremely large lists or tables, consider windowing libraries (though `FlashList` covers a lot). If you have a very long text or graph, perhaps paginate it.
- **Caching and persistence:** Use caching for expensive fetches – e.g., caching API responses in memory or disk (with something like `Redux Persist` or `react-query`'s cache) to avoid repetitive heavy loads.
- **Remove console logs and dev stuff in production:** They can slow down execution (especially `console.log` on Android is notoriously slow if logging a lot). Ensure you strip them in release (Metro does remove console.* if you configure it, or use Babel plugin to remove dev code).

By applying these optimizations, you'll achieve faster load times, smoother animations, and lower memory usage, resulting in a snappy, high-performance app. As an example of impact: enabling Hermes often yields a noticeable drop in app startup time and memory, and lazy-loading screens can reduce Time To Interactive on app launch significantly by not loading all code upfront ⁷¹ ⁷³. Optimizing performance is an ongoing process – continuously profile your app (especially on lower-end Android devices) to find bottlenecks.

Deployment and Releasing Apps (App Store & Play Store)

After development and testing, you'll need to **build and distribute** your React Native app on Apple's App Store (for iOS) and Google Play Store (for Android). The process differs slightly between **Expo managed apps** and **bare React Native apps**, but many steps (like creating store listings and certificates) are similar:

- **Apple App Store (iOS):** To publish on the App Store, you must be enrolled in the Apple Developer Program (annual \$99) ⁸⁰. You'll use **App Store Connect** to manage your app listing (name, description, screenshots, etc.) and to upload your binary. For **bare RN CLI apps**, you generate an iOS **Archive** (IPA file) via Xcode or CLI:
- In Xcode, increment your app version and build number in the project settings (iOS requires unique version+build for each release). Select *Any iOS Device (arm64)* and do `Product -> Archive`. After archiving, use the **Organizer** to Distribute the app (this uses either automatic signing or your provisioning profile to code-sign the app) to the App Store ⁸¹ ⁸². Alternatively, you can use **fastlane** with `gym` and `deliver` to automate building and uploading.
- Once uploaded, the build appears in App Store Connect under your app's TestFlight section. Initially, you likely distribute to **TestFlight** (Apple's beta testing platform) – you can invite internal or external testers. TestFlight helps ensure your app passes basic validation before full release.
- Finally, you submit the app for **App Store Review**. Fill out compliance information, privacy policy URL, etc., in App Store Connect (Apple is strict about providing a privacy policy and describing usage of sensitive APIs). Then hit "Submit for Review". Apple will manually review your app for guidelines compliance (this can take a day or a few days). Upon approval, you can release the app to the public.

For **Expo managed apps**, the recommended route is to use **EAS Build** and **EAS Submit**: - Run `eas build -p ios` (with a proper `eas.json` config if needed) to build the iOS app in Expo's cloud. EAS handles provisioning profiles and signing – you'll be prompted to log in to Apple and either let Expo create certificates or upload your own ⁸³. Once the cloud build is done, you get an IPA. - Then run `eas submit -p ios --latest` to upload that IPA to App Store Connect ⁸². EAS Submit can use an App Store Connect API key you provide to authenticate and upload directly, streamlining the process. - After

upload, the steps are the same: the build shows up in App Store Connect, and you proceed with TestFlight testing and then app review.

- **Google Play Store (Android):** You need a Google Play Developer account (a one-time \ \$25 fee) ⁸⁴ . On the Play Console, create a new application listing and provide the required details (app name, description, category, graphics assets like icons and screenshots). Android apps are distributed as **APK** or more commonly **AAB** (Android App Bundle) now. For **bare RN CLI apps**:
 - Ensure you have a release keystore generated (using `keytool`) and configured in `android/app/build.gradle`. Update your app's `versionName` and `versionCode` in `android/app/build.gradle` for each release.
 - Build the release AAB by running `./gradlew bundleRelease` (or APK with `assembleRelease`). This will produce an `app-release.aab` in `android/app/build/outputs/bundle/release/`.
 - In Play Console, you typically start with an **Internal Testing** or **Closed Testing** track. Upload the AAB under that track, add testers (e.g., by email or create an open beta link) ⁸⁵ ⁸⁶ . Google will process the AAB (it does an integrity check and provides pre-launch reports). You might need to fill in the **App Content** section (like content rating questionnaire, privacy policy, and whether your app targets children or contains ads) before releasing ⁸⁷ ⁸⁸ .
 - Once satisfied, promote the release to **Production** to launch it to all users. Google Play review is generally faster and more automated than Apple's, but new apps can still take a few days for initial approval.

For **Expo apps**, use `eas build -p android` to get an AAB. Expo will either use your supplied keystore or generate one for you (it will prompt on first build; Expo can store the keystore securely for you). After the cloud build, run `eas submit -p android --latest` to upload to Play Store. You'll need to have set up Google API access (service account JSON) for EAS to upload, or you can manually upload the AAB via Play Console's web UI ⁸⁹ .

Play Console requires at least one **in-app privacy policy URL** if your app requests sensitive permissions, and by 2025, you must declare how user data is used (Data Safety section). Ensure you fill those out to avoid rejection ⁸⁸ .

- **Versioning and Updates:** Follow semantic versioning for marketing version (e.g., 1.0, 1.1.0) and increment the internal build numbers properly. iOS uses `CFBundleVersion` (an integer usually) as build number, Android uses `versionCode` (integer). Each app store upload must have a higher number than the previous. Expo managed apps will auto-increment if you configure it, or you can set it via `app.json/app.config.js` for each release.
- **Post-release Updates (OTA):** One huge advantage for RN apps is the ability to push over-the-air JavaScript updates for quick fixes. For Expo apps, **EAS Update** is the official solution. By installing the `expo-updates` module (Expo does this by default) and publishing updates, you can instantly deploy JS bundle changes (and asset changes) to users who already have the app ⁹ ⁹⁰ . For example, if you find a typo or need to toggle a feature flag, you can run `eas update --branch production` (after configuring EAS Update) and users' apps will download the update on next launch (or even in the background). Expo's updates service ensures only non-native changes are updated – if you change anything that requires a native build (like adding a new library with native code), that still requires a full app store release.

For bare RN apps, third-party solutions like **Microsoft CodePush** (part of App Center) have traditionally been used for OTA updates. CodePush allows you to deploy JS bundle diffs to users, similar to EAS Update. However, note that Microsoft has indicated CodePush may not be maintained in the far future ⁹¹, and alternative services like **AppCenter's new Distribution** or community solutions (e.g., Electrode OTA, or the emerging Expo OTA for bare apps) are being explored. As of 2025, CodePush still works and many companies use it to hotfix production without waiting for store review. If using CodePush, integrate the `react-native-code-push` SDK and configure app keys for each platform. Then use the CLI to release updates.

Important: OTA updates must comply with store policies – you cannot add entirely new features or change the app's primary functionality via OTA without a store review. Apple and Google allow OTA for bug fixes and minor changes, but anything that would alter the app's binary capabilities or entitlements must go through the store. Always be cautious and follow guidelines.

- **Continuous Deployment:** In production scenarios, you might integrate CI/CD so that every merge to `main` triggers a build and maybe even submits to TestFlight/Internal testing. Expo's **EAS Workflows** (CI/CD service) can automate build, test (even run Detox tests on the cloud with services like Maestro) ⁹², and deployment steps via a declarative config (YAML) ⁹³ ⁹⁴. Alternatively, use GitHub Actions or CircleCI for custom pipelines – e.g., run tests, then run `eas build` or use fastlane to build, and automatically upload to TestFlight for QA. Automating this reduces manual errors and speeds up your release cycle.
- **App Store Optimization & Listing:** While not a development practice per se, ensure your **store listing** is polished – include high-quality screenshots (you can take them via emulators or attach a physical device), possibly an app preview video, and localized descriptions if targeting multiple regions. This will improve your app's discoverability and user appeal. On iOS, you might need to provide an **ICON** for every required size and a **Splash screen storyboard** (Expo handles this via `app.json`, and bare apps via Assets.xcassets). On Android, provide adaptive icons for different shapes. These are typically set up during development but are crucial for release quality.
- **Deployment Best Practices:**
 - Test your release builds thoroughly (especially if you use any config that differs in release, like different API endpoints or feature flags). Use TestFlight and Google's internal testing to catch last-minute issues.
 - Monitor crashes and analytics from day one. Use tools like **Sentry or Firebase Crashlytics** for crash reporting in production – they'll help you catch any platform-specific crashes quickly. Expo provides integration for Sentry via config plugins, and for bare RN you can add `@react-native-firebase/crashlytics` or Sentry SDK.
 - Gradual rollouts: Google Play allows staged rollouts (e.g., release to 10% of users, then ramp up) which can be wise for major updates. Apple doesn't have an automatic staged rollout, but you can release to TestFlight external testers which is somewhat similar.
 - **EAS Metadata / Fastlane metadata:** to automate store listing updates, you can store your app metadata (descriptions, etc.) in your repo and use tools to push them. Expo introduced EAS Metadata for this, and Fastlane has deliver & supply for iOS/Android metadata.

By following these deployment practices, you ensure a smoother path from development to your users' devices. In summary, **Expo's EAS** can significantly streamline the app store submission process (handling the tedious parts like code signing and uploads for you ⁸), whereas **bare workflow** gives you flexibility to integrate with existing enterprise CI systems. Either way, stay organized with your provisioning profiles, keystores, and app identifiers. Once your app is live, leverage OTA updates for minor fixes ⁹ and keep an eye on user feedback and crash reports to iterate quickly.

Integrating Backend Services (Firebase, APIs, GraphQL, etc.)

Most mobile apps need to communicate with backend services or use cloud functionality. React Native, being JavaScript, has access to the vast ecosystem of web libraries for HTTP networking, as well as specialized SDKs for services like Firebase. Here's how to approach backend integration:

- **Calling RESTful APIs:** React Native provides the **Fetch API** globally (same as in browsers) for network requests ⁹⁵. You can use `fetch()` to call your REST endpoints, or use libraries like **Axios** for a richer feature set (interceptors, easier JSON handling, etc.). The pattern is similar to web React apps: call the API (usually in a `useEffect` or in an event handler), then update state with the response. For example:

```
useEffect(() => {
  let cancel = false;
  fetch('https://api.example.com/products')
    .then(res => res.json())
    .then(data => { if(!cancel) setProducts(data); })
    .catch(err => console.error(err));
  return () => { cancel = true; }; // cancel on unmount
}, []);
```

Ensure you handle unmounted component scenarios (as above) to avoid setting state on unmounted components. For better data management, consider using **TanStack Query (React Query)** or **RTK Query**. These libraries handle caching, background refetching, and updating your UI when data changes with far less boilerplate than manual fetch. In 2025, React Query (now called TanStack Query) is widely used to manage server state in RN apps ⁹⁶ – it provides hooks like `useQuery` and `useMutation` that automatically manage loading states, caching, and retries. Redux Toolkit's `createApi` (RTK Query) similarly can define endpoints and generate hooks to use in components, which is great if you're already using Redux.

- **Using GraphQL APIs:** If your backend provides a GraphQL endpoint, using **Apollo Client** or **Relay** can greatly simplify data handling. **Apollo Client** is very popular for React Native (as it is for React web) due to its ease of setup and powerful cache. To integrate:
 - Install Apollo (`npm install @apollo/client graphql`).
 - Initialize an ApolloClient with your server's URI and an `InMemoryCache` ⁹⁷ ⁹⁸.
 - Use `<ApolloProvider client={client}>` to wrap your app so any component can use GraphQL hooks.

Example setup of Apollo in RN:

```
import { ApolloClient, InMemoryCache, ApolloProvider } from '@apollo/client';
const client = new ApolloClient({
  uri: 'https://your-api.com/graphql',
  cache: new InMemoryCache()
});
export default function App() {
  return (
    <ApolloProvider client={client}>
      <NavigationContainer>{/* ... */}</NavigationContainer>
    </ApolloProvider>
  );
}
```

⁹⁸ ⁹⁹ . Then inside your components you can use `useQuery` or `useMutation` hooks (from `@apollo/client`) to fetch data. Apollo will handle caching results (so if you query the same data again it can come from cache), and also allows local state management via the cache. Best practices include defining GraphQL **fragments** for reusability and using GraphQL's subscription support if you need real-time updates (requires setting up WebSocket link).

If you have extremely high performance requirements and complex data graphs, **Relay** is another GraphQL client that Facebook uses internally – it's more complex but highly optimized. However, Apollo Client is generally sufficient and easier to adopt. The medium article by Mesut K. on using GraphQL with RN emphasizes writing efficient queries (using fragments and pagination) to avoid over-fetching ¹⁰⁰ ¹⁰¹ , which is crucial for mobile performance. One benefit of GraphQL in mobile is you can request exactly the data needed for a screen, reducing payload size and number of requests (e.g., one round-trip to get a feed list and user details in one query, instead of multiple REST calls).

- **Firestore Integration:** Firebase offers a suite of backend services (Auth, Firestore database, Cloud Functions, Cloud Storage, Push Notifications via FCM, Analytics, Crashlytics, etc.). **React Native Firebase** (RNFirebase) is the official set of libraries that wrap Firebase's native SDKs for RN ¹⁰² . It includes packages like `@react-native-firebase/auth`, `firestore`, `messaging`, etc. To use RNFirebase in a bare RN app, you install the packages and run the native setup (adding Google services config files and Gradle changes). In Expo's managed workflow, you **can** still use RNFirebase by creating a development build with config plugins ¹⁰³ ¹⁰⁴ – Expo now supports it, though you can't use it in the Expo Go app since that doesn't include Firebase native SDKs.

For example, to add Firebase Auth and Firestore to an Expo app: 1. Install the packages: `npx expo install @react-native-firebase/app @react-native-firebase/auth @react-native-firebase/firestore`. 2. Configure the native modules via Expo's config plugin in `app.json` or `app.config.js` (the RNFirebase docs detail which config plugin to add for each module) ¹⁰⁴ ¹⁰⁵ . 3. Rebuild your Expo dev client (`eas build -p ios --profile development` or run in Expo prebuild mode) to get the native code.

Once set up, usage is straightforward: e.g., `import auth from '@react-native-firebase/auth';` `auth().signInWithEmailAndPassword(email, pass)` returns a Promise for sign-in. Similarly, `firestore().collection('users').get()` fetches documents. RNFirestore mirrors the web Firebase JS API in many ways but calls native under the hood for efficiency.

If you prefer not to use the native Firebase SDKs, you can sometimes use Firebase's **Web SDK** (especially for analytics or basic auth). For example, in pure JS (no native) you could include the Firebase web libraries via npm and initialize. However, not all features work (analytics might, but notifications definitely require native FCM). The RNFirestore approach is more robust for device features and is the recommended way for production ¹⁰² ¹⁰³.

Common Firebase use-cases in RN: - **Push Notifications:** Use `@react-native-firebase/messaging` to get an FCM token and handle notifications. On iOS, request permission with `messaging().requestPermission()`. Expo alternatively has `expo-notifications` which can integrate with Firebase's FCM behind the scenes for Expo apps. - **Authentication:** `@react-native-firebase/auth` supports email/password, phone auth (with SMS), and OAuth providers (Google, Facebook, etc.). Note that for OAuth in RN, you often integrate with deep linking or SDKs (Expo's `AuthSession` or `GoogleSignIn` libraries can complement Firebase Auth). - **Realtime DB / Firestore:** You can use `onSnapshot` listeners in Firestore for real-time updates. Be mindful of data usage and unsubscribing listeners when components unmount to avoid leaks. - **Crashlytics & Analytics:** With `@react-native-firebase/crashlytics` and `analytics`, you can get crash reports and track user behavior events, which is invaluable post-deployment.

Using Firebase can drastically speed up development since you avoid writing a lot of backend code, but remember to secure your Firebase project (Firestore rules, etc.) because mobile apps can be decompiled. Always test on both OS for any differences (e.g., phone auth flows differ on iOS vs Android due to SMS auto-read behaviors).

- **Other Backend Services:**

- **REST APIs / Backend-as-a-Service:** If not using Firebase, you might use AWS Amplify or Appwrite or Supabase for backend. These have their own RN support (Amplify has React Native libraries, Supabase uses the web JS client which works in RN with polyfills).

- **GraphQL Backend:** If you use GraphQL, services like Apollo GraphOS or Hasura can host the backend. Your RN app doesn't change, it's all in how the backend is structured.

- **Socket Connections:** For real-time communication (chat apps, live updates), you can use WebSockets. Libraries like **Socket.IO** have RN clients, or use the WebSocket API directly (`new WebSocket('wss://...')`). Another approach is using GraphQL subscriptions or Firebase realtime DB for real-time data.

- **Offline Data & Caching:** Tools like **SQLite** (via `react-native-sqlite-storage` or WatermelonDB for complex offline sync) or **AsyncStorage** for simple key-value storage help manage offline scenarios. If your app needs to work offline, consider strategies to cache server responses (e.g., using react-query's persistence, or writing to SQLite) and to sync when back online.

- **Security and Environment Config:** Never put sensitive secrets (API keys for secure services, etc.) directly in the JS code that gets shipped. For RN, you can use **.env files** and a library like `react-native-dotenv` or `expo-constants` for managed apps to keep config out of source control and

possibly use native configuration for truly sensitive items (like API secret keys should reside on a server if possible). Firebase keys, etc., are okay to include (they are not true secrets, just config), but treat anything that would allow unauthorized access carefully (for example, don't trust the client to enforce data rules – always enforce on server/Firebase security rules).

- **Example: Apollo + Redux Toolkit Query + Firebase** – you might wonder if these can coexist. Yes: for instance, you could use RTK Query for some REST endpoints, Apollo for a GraphQL service, and Firebase for auth & notifications. Just keep concerns separate (maybe your Redux store gets a slice for some data that isn't covered by Apollo). It's not uncommon for large apps to use multiple data fetching methods.

In essence, React Native doesn't limit your backend choices: anything you can call over HTTP or have a JS SDK for, you can use. The **official Apollo docs** have a section for integrating with React Native (mostly identical to React web usage) ¹⁰⁶. Firebase's official RN docs guide you through each service's setup in detail. Prioritize using **official or well-maintained libraries** for these integrations (e.g., the community Firebase package, Apollo client, etc.) to ensure compatibility and support.

By following these guidelines and using the appropriate tools, you can connect your React Native app to powerful backend services – enabling features like real-time data, cloud storage, authentication, and more – while maintaining performance and a good developer experience. Always test network interactions on device (including in poor network conditions or offline) to ensure your app handles them gracefully (e.g., use timeout/retry strategies and user-friendly error messages for network failures).

By embracing these modern practices – from leveraging Reanimated for buttery animations to using EAS for streamlined deployments and applying clean architecture principles – you'll be well-equipped to build a React Native app that is **fast, beautiful, and maintainable** in 2025. Remember to keep learning from official docs and community updates, as the React Native ecosystem continues to evolve with better tools and patterns. Happy coding!

Sources: The information and recommendations above are based on React Native's official documentation, community best-practices, and widely respected resources ¹⁰⁷ ²² ⁴⁵ ⁷¹ ⁹⁸, ensuring you're getting the most up-to-date guidance for 2025 development.

¹ ² ³ ⁶ ⁷ ¹¹ ¹³ ¹⁴ ¹⁵ ¹⁰⁷ Expo vs React Native CLI: Key Differences Explained

<https://flatirons.com/blog/expo-vs-react-native/>

⁴ ⁵ ⁸ ¹² ¹⁶ Expo in 2025: Unlocking Production-Ready Power for Scalable React Native Apps | Stackademic

<https://blog.stackademic.com/expo-in-2025-unlocking-production-ready-power-for-scalable-react-native-apps-5ac78e02e374?gi=b160e2452c27>

⁹ ¹⁰ ⁹⁰ EAS Update - Expo Documentation

<https://docs.expo.dev/eas-update/introduction/>

¹⁷ ¹⁸ ²⁰ ⁷⁹ React Native's New Architecture - Expo Documentation

<https://docs.expo.dev/guides/new-architecture/>

19 **GitHub - software-mansion/react-native-reanimated: React Native's Animated library reimplemented**
<https://github.com/software-mansion/react-native-reanimated>

21 22 23 24 25 26 27 28 29 30 31 32 **React Native Reanimated 3: The Ultimate Guide to High-Performance Animations in 2025 - DEV Community**
<https://dev.to/erenelagz/react-native-reanimated-3-the-ultimate-guide-to-high-performance-animations-in-2025-4ae4>

33 42 43 **State of React Native 2024: Graphics and animations**
<https://results.stateofreactnative.com/en-US/animations/>

34 35 36 **Welcome to Moti | Moti**
<https://moti.fyi/>

37 **Moti vs. Reanimated**
<https://moti.fyi/reanimated>

38 **Optimizing Lottie Animations in React Native with the .lottie Format**
<https://medium.com/@mukkadeepak/optimizing-lottie-animations-in-react-native-with-the-lottie-format-8f7a31ff53ed>

39 **Boost Your React Native App Start Time: Stop Shipping Lottie JSON ...**
<https://dev.to/retyui/boost-your-react-native-app-start-time-stop-shipping-lottie-json-incorrectly-2074>

40 41 **Advanced React Native Animation Techniques Guide for 2025 | Viewlytics**
<https://viewlytics.ai/blog/react-native-advanced-animations-guide>

44 45 70 78 **React Native Development Best Practices for Modern Mobile Apps | by Jignen Pandya | Medium**
<https://medium.com/@jignen.pandya/react-native-development-best-practices-for-modern-mobile-apps-7487c8cb5c41>

46 47 48 49 **Architecture pattern for React Native applications | by Andrei Lomtev | Stackademic**
<https://blog.stackademic.com/architecture-pattern-for-react-native-applications-906fff65ebf8?gi=e0c6562f4cca>

50 51 52 53 54 55 56 57 58 59 **Using TypeScript · React Native**
<https://reactnative.dev/docs/typescript>

60 61 62 63 64 65 66 68 **Testing · React Native**
<https://reactnative.dev/docs/testing-overview>

67 **Detox**
<https://wix.github.io/Detox/>

69 92 **Cut CI from 20 minutes to 6 with EAS Workflows and EAS Updates**
<https://shift.infinite.red/cut-ci-from-20-minutes-to-6-with-eas-workflows-and-eas-updates-8d928fe5dcfc>

71 72 73 74 75 76 **Optimizing JavaScript loading · React Native**
<https://reactnative.dev/docs/optimizing-javascript-loading>

77 **How can I improve React Native app performance on Android?**
https://www.reddit.com/r/reactnative/comments/1kz260d/how_can_i_improve_react_native_app_performance_on/

80 81 82 83 84 85 86 87 88 89 **How to Publish Expo React Native App to iOS and Android in 2025**
<https://pagepro.co/blog/publishing-expo-react-native-app-to-ios-and-android/>

91 **What to do without CodePush - Expo**
<https://expo.dev/blog/what-to-do-without-codepush>

93 94 Get started with EAS Workflows - Expo Documentation

<https://docs.expo.dev/eas/workflows/get-started/>

95 Beginner's Guide to React Query - Refine dev

<https://refine.dev/blog/react-query-guide/>

96 Why You Should Learn React Query in 2025 — Goodbye Redux for ...

<https://medium.com/@theNewGenCoder/%EF%B8%8F-why-you-should-learn-react-query-in-2025-goodbye-redux-for-data-fetching-2912e08f1264>

97 98 99 100 101 106 Using GraphQL with React Native: Best Practices for Efficient Data Fetching | by Mesut KILINCASLAN | Medium

<https://medium.com/@mesutas.dev/using-graphql-with-react-native-best-practices-for-efficient-data-fetching-a0a42d01c671>

102 103 104 105 React Native Firebase | React Native Firebase

<https://rnfirebase.io/>